

IE 643 COURSE PROJECT · STAGE-WISE REPORT

Unsupervised Domain Adaptation for Forecasting Multivariate Time-Series Data

A lightweight forecaster trained on one domain (hilly terrain, IT stocks, transformer 1) and adapted without target labels to another (plain terrain, pharma stocks, transformer 2) with CARE-DA.

Team: [Team Name]

[Member 1 Name] — [Roll No.]

[Member 2 Name] — [Roll No.]

[Member 3 Name] — [Roll No.]

[Member 4 Name] — [Roll No.]

Draft version · 20 September 2026

Contents

Abstract	3
1. Introduction and problem statement.....	4
1.1 Formal setting.....	4
1.2 Mapping of project tasks to this report.....	4
2. Stage 1 — Background study	5
2.1 Lightweight forecasting models	5
2.2 Theory of domain adaptation.....	5
2.3 Families of domain-adaptation methods	5
2.4 Domain adaptation for time series.....	5
3. Stage 2 — Dataset collection and preparation.....	7
3.1 Extraction steps	7
3.2 Curation and preparation	7
3.3 Evidence of domain shift	8
4. Stage 3 — Lightweight forecaster for Domain 1	9
4.1 Source training	9
5. Stage 4 — Adaptation strategy: CARE-DA	10
5.1 Algorithm	10
6. Stage 5 — Loss function design	11
7. Stage 6 — Performance metrics.....	12
7.1 Forecast accuracy	12
7.2 Adaptation efficiency.....	12
8. Stage 7 — Experiments.....	13
8.1 Protocol	13
8.2 Main results.....	13
8.3 Findings.....	14
8.4 Adaptation behaviour (logs and plots)	15
8.5 Post-hoc sensitivity.....	18
9. Stage 8 — Interface methodology.....	19
9.1 Design	19
9.2 How to run.....	20
10. Discussion and limitations	21
11. Future work and roadmap.....	22
12. Code, documentation and reproducibility	23
References.....	24

Abstract

We study unsupervised domain adaptation (UDA) for multivariate time-series forecasting: a model trained on one domain must forecast well on a related domain for which no future (label) values are used during adaptation. We curated three benchmarks with distinct source and target domains: hourly weather in hilly Shimla vs plain New Delhi (Open-Meteo ERA5), daily NSE IT vs pharma stocks (Yahoo Finance) and two electricity transformers (ETTh1 vs ETTh2).

We built LiteTCN, a 0.13–0.38 M-parameter forecaster that combines reversible instance normalisation, a dilated convolutional encoder, a DLinear-style linear path and a masked-reconstruction head. We then designed CARE-DA, an adaptation objective combining source forecasting loss, multi-kernel MMD and CORAL alignment of latent features, masked reconstruction on target windows, and an L2-SP anchor to the source weights.

Across three seeds, the designed CARE-DA objective reduces target-domain MSE relative to the unadapted model by 3.6% (climate) and 3.3% (energy), where the supervised fine-tuning oracle gains 8.5% and 3.0%; it leaves finance unchanged. Ablations and a post-hoc sensitivity study show that feature alignment is the effective component: the variant without the self-supervised term improves on all three benchmarks (−7.8%, −6.4%, −0.6%) and on energy beats the label-using oracle. The masked-reconstruction term, contrary to our hypothesis, consistently hurts. Adversarial (DANN) and CORAL-only adaptation caused negative transfer on all benchmarks. A Streamlit interface lets users upload Domain 1 or Domain 2 CSVs and inspect inputs, actual values, forecasts and metrics.

1. Introduction and problem statement

A forecasting model is typically trained on historical data from one setting and then deployed in another. When the statistical behaviour of the series changes between the two settings (a **domain shift**), the accuracy of the model degrades. The project brief illustrates this with temperature forecasting: a model built for a hilly region must be reused for a plain region, and similarly a model for an IT-sector stock must be reused for a pharma-sector stock.

We study the **unsupervised** version of this problem. In Domain 1 (the source) we have full multivariate histories and train a forecaster. In Domain 2 (the target) we may use input windows, but no future values are used for training the adapted model or for choosing its checkpoint. The goal is to reduce the forecasting error on Domain 2 as much as possible relative to (i) simply reusing the source model and (ii) an oracle that is fine-tuned with target labels.

1.1 Formal setting

Let $x \in \mathbb{R}^{(L \times C)}$ be a look-back window of L time steps and C variables and $y \in \mathbb{R}^{(H \times C)}$ the next H steps. The source domain provides pairs $D_S = \{(x_i, y_i)\} \sim P_S$; the target provides inputs $D_T = \{x_j\} \sim P_T(x)$ with $P_T(x, y) \neq P_S(x, y)$. Starting from a source model f_{θ_S} we seek parameters θ' that minimise the target risk:

$$\theta' = \operatorname{argmin}_{\theta} E_{(x,y) \sim P_T} [(1/H) \sum_C \| f_{\theta}(x) - y \|^2] \text{ using only } D_S \text{ and } D_T$$

1.2 Mapping of project tasks to this report

Project requirement	Where addressed	Status
Collect ≥ 3 time-series datasets with distinct domains	Section 3 (climate, finance, energy)	Done (to be verified with TAs)
Lightweight forecasting model on Domain 1	Section 4 (LiteTCN, ~0.13–0.38 M parameters)	Done
Adaptation strategy	Section 5 (CARE-DA)	Done (v1)
Implement and record adaptation behaviour (logs, plots)	Section 8.4, results/<ds>/histories.json, train_log.txt	Done
Design a loss function for adaptation	Section 6	Done
Explore performance metrics	Section 7	Done
Interfaces for Domain 1 and Domain 2 CSV input + metrics	Section 9 (Streamlit app)	Done (v1)

2. Stage 1 — Background study

We first studied (a) lightweight forecasting architectures, (b) the theory of domain adaptation, (c) families of adaptation methods and (d) prior work on domain adaptation for time series. The purpose was to find components that are cheap, well understood, and suited to **regression** (forecasting) rather than classification.

2.1 Lightweight forecasting models

- **DLinear** [14] shows that a single linear layer mapping the look-back window to the horizon (after a trend/seasonal decomposition) is competitive with Transformer forecasters on long-horizon benchmarks. We keep such a linear path inside our model as a strong, stable baseline component.
- **Temporal Convolutional Networks** [16] use stacked dilated 1-D convolutions, which give a large receptive field with few parameters and fully parallel training. We use a strided, dilated convolutional encoder to produce a compact latent representation that can be aligned across domains.
- **PatchTST** [15] tokenises the series into patches and uses channel-independent Transformers. It is state of the art but heavier; we list it as a candidate encoder for later stages.
- **RevIN** [13] normalises each input window by its own mean and standard deviation and de-normalises the output. It directly removes per-window level and scale shift, the simplest form of domain shift.

2.2 Theory of domain adaptation

Ben-David et al. [1] bound the target error of a hypothesis h by

$$\varepsilon_T(h) \leq \varepsilon_S(h) + \frac{1}{2} d_{\text{H}\Delta\text{H}}(D_S, D_T) + \lambda^*$$

where ε_S is the source error, $d_{\text{H}\Delta\text{H}}$ measures how distinguishable the two domains are to the hypothesis class, and λ^* is the error of the best joint hypothesis. The bound motivates three design goals that we map directly onto the terms of our loss: keep the source error low; reduce the divergence between source and target representations; and avoid representations for which no good joint predictor exists (i.e. do not align at the cost of losing forecast-relevant information).

2.3 Families of domain-adaptation methods

Family	Idea	Representative work	Use in this project
Discrepancy-based	Minimise a statistical distance between feature distributions	MMD / DAN [4,5], Deep CORAL [3]	Alignment terms of CARE-DA; baselines
Adversarial	Fool a domain classifier via gradient reversal	DANN [2], CoDATS [9]	Baseline
Self-supervised / test-time training	Auxiliary label-free task on target data	TTT [7]	Masked-reconstruction term
Parameter regularisation	Penalise distance to pre-trained weights	L2-SP [6]	Elastic anchor term
Normalisation	Remove instance or domain statistics	RevIN [13], AdaBN	RevIN + per-domain scaling

2.4 Domain adaptation for time series

- **CoDATS** [9]: adversarial adaptation of 1-D CNNs for multi-source sensor classification.

- **SASA** [10]: aligns the sparse associative structure between variables, i.e. which variables influence which.
- **DAF** [8]: an attention-sharing Transformer for forecasting across domains; it assumes some labeled target data.
- **CLUDA** [11]: contrastive learning with nearest-neighbour alignment for time-series UDA.
- **RAINCOAT** [12]: aligns both time- and frequency-domain features and handles feature and label shift.

Take-away. Most time-series UDA work targets classification; forecasting-oriented DA usually needs labeled target data; and heavy backbones are expensive to adapt repeatedly. We therefore aim for label-free adaptation of a lightweight multivariate forecaster and evaluate how much of the supervised gap it closes.

3. Stage 2 — Dataset collection and preparation

We selected three public sources so that every step is reproducible with one command (`python -m src.data_download`). Each benchmark provides two domains with **identical variables** but different underlying behaviour.

Benchmark	Domain 1 (source)	Domain 2 (target)	Variables	Span / frequency	L → H
Climate	Shimla (31.10°N 77.17°E, 2195 m, hilly)	New Delhi (28.61°N 77.21°E, 214 m, plain)	temperature_2m, relative_humidity_2m, dew_point_2m, surface_pressure, wind_speed_10m, cloud_cover, shortwave_radiation	2021-01-01 to 2024-12-31, hourly (35064 rows each)	168 → 24
Finance	NSE IT: TCS, INFY, WIPRO, HCLTECH, TECHM	NSE Pharma: SUNPHARMA, DRREDDY, CIPLA, DIVISLAB, LUPIN	log_return, hl_range, gap, log_volume	2012-01-03 to 2025-06-27, trading days (3322 per stock)	60 → 5
Energy	ETTh1 (transformer, county 1)	ETTh2 (transformer, county 2)	HUFL, HULL, MUFL, MULL, LUFL, LULL, OT	2016-07-01 to 2018-06-26, hourly (17420 rows each)	168 → 24

3.1 Extraction steps

Climate (Open-Meteo historical weather API, ERA5 reanalysis)

- Endpoint <https://archive-api.open-meteo.com/v1/archive> with latitude/longitude of each site, start_date=2021-01-01, end_date=2024-12-31, timezone=Asia/Kolkata.
- Hourly variables: temperature_2m, relative_humidity_2m, dew_point_2m, surface_pressure, wind_speed_10m, cloud_cover, shortwave_radiation.
- Raw JSON responses are stored in `data/raw/climate/`; the returned elevation (2195 m vs 214 m) confirms the hilly-vs-plain contrast.

Finance (Yahoo Finance via `yfinance`)

- Daily split/dividend-adjusted OHLCV for five NSE IT and five NSE pharma stocks, 2012-01-01 to 2025-06-30; raw files in `data/raw/finance/`.
- Prices are non-stationary and on very different scales across stocks, so we derive four stationary features per day: $\log_return = 100 \cdot \ln(C_t/C_{t-1})$; $hl_range = 100 \cdot \ln(H_t/L_t)$ (a volatility proxy); $gap = 100 \cdot \ln(O_t/C_{t-1})$; $\log_volume = \ln(1+V_t)$.
- Days with zero volume or invalid ranges are removed. Each stock is a separate series (windows never cross stock boundaries); the five series are pooled per sector.

Energy (ETT benchmark [17])

- ETTh1.csv and ETTh2.csv downloaded from <https://github.com/zhouhaoyi/ETDataset>: hourly load features (HUFL, HULL, MUFL, MULL, LUFL, LULL) and oil temperature (OT) of two electricity transformers in different counties.
- The standard 12/4/4-month (60/20/20) chronological split is used.

3.2 Curation and preparation

- **Missing values:** linear interpolation along time (the climate API returned complete series; the step is kept for user-uploaded CSVs).

- **Chronological split** per series: 70/10/20 (train/validation/test) for climate and finance, 60/20/20 for ETT. Validation and test segments are prefixed with L steps of history so that the first test window has a full look-back; no future information leaks backwards.
- **Per-domain z-scoring**: each domain is standardised with the mean and standard deviation of its own training segment. For the target this needs only unlabeled inputs, which is allowed in UDA. Metrics are computed in this standardised space so that variables with different units contribute equally.
- **Windowing**: sliding windows of L + H steps (stride 2 for climate training to limit memory, stride 1 otherwise). All C variables are both inputs and forecast targets (multivariate-to-multivariate).

3.3 Evidence of domain shift

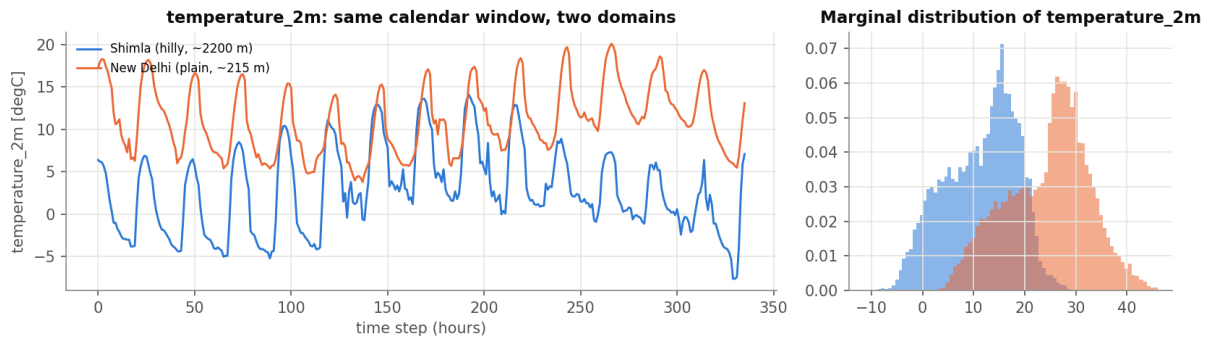


Figure 1. Temperature in Shimla vs New Delhi: mean 11.4 °C vs 24.6 °C; surface pressure 783 vs 984 hPa.

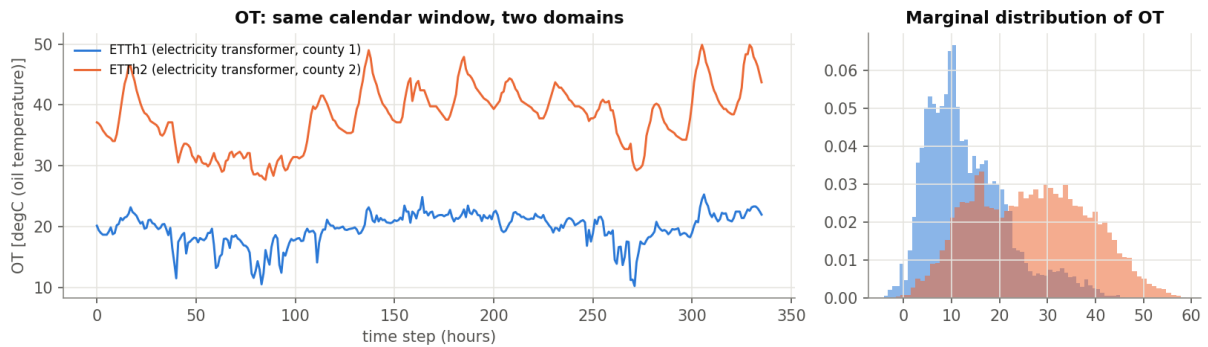


Figure 2. Oil temperature of the two ETT transformers (mean 13.3 vs 26.6 °C).

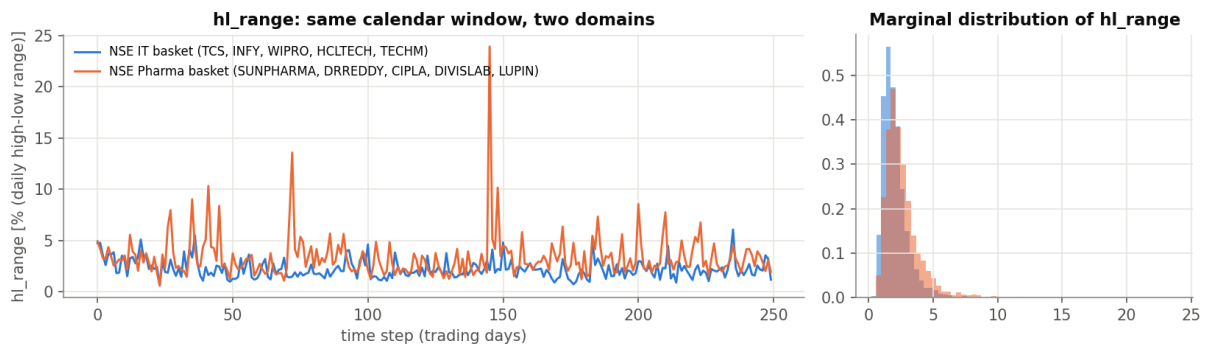


Figure 3. Daily high-low range (volatility proxy) for an IT and a pharma stock (sector means 2.33% vs 2.57%).

We distinguish three kinds of shift: (i) **marginal/level shift** (different means and scales), which per-domain scaling and RevIN remove; (ii) **dynamics or covariate shift** (different daily amplitude, regimes and cross-variable correlations), which changes what the encoder sees; and (iii) **conditional shift**, where the same recent history leads to a different future. Types (ii) and (iii) are what the adaptation strategy must address.

4. Stage 3 — Lightweight forecaster for Domain 1

The **LiteTCN-Forecaster** (src/model.py) has 381,606 parameters for $L = 168$, $C = 7$ (≈ 0.13 M for finance). It was designed for adaptation: it exposes a latent vector that can be aligned between domains, and it has an auxiliary head that provides a label-free learning signal in the target domain.

- **RevIN** normalises each input window per variable (learnable affine), and the output is de-normalised with the same statistics.
- **Encoder**: three Conv1d layers (kernel 5; stride 2; dilation 1, 2, 2; GELU), widths 32-64-64, followed by flatten $\rightarrow$ dropout (0.1) $\rightarrow$ linear $\rightarrow$ GELU, giving a 128-d latent vector z .
- **Forecast head**: linear $z \rightarrow H \cdot C$, added to a **linear temporal path** (DLinear-style $L \rightarrow H$ map shared across variables) applied to the normalised input.
- **Reconstruction head**: linear $z \rightarrow L \cdot C$. During training, 30% of the time steps of the input are masked and must be reconstructed from z (masked-reconstruction pretext task).

4.1 Source training

The source model is trained with MSE on the forecast plus $0.5 \times$ the masked-reconstruction loss (so that the reconstruction head is already meaningful before adaptation), using AdamW (lr $1e-3$, weight decay $1e-4$), a cosine schedule, batch size 256, gradient clipping at 1.0 and 20 epochs. The checkpoint with the lowest source-validation MSE is kept. Source-domain test MSE (z-scored): Climate 0.227, Energy 0.379, Finance 0.615.

5. Stage 4 — Adaptation strategy: CARE-DA

CARE-DA (Correlation-Aligned, Reconstruction-regularised, Elastic-anchored Domain Adaptation) fine-tunes a copy of the source model on mixed batches: a labeled source batch (x_s, y_s) and an unlabeled target batch x_t . The shared network produces latents z_s and z_t . Four goals, each motivated by the bound in Section 2.2, are optimised jointly:

- **Keep source skill** (ϵ_S): supervised forecasting loss on the source batch.
- **Close the domain gap** ($d_{H\Delta H}$): align the distributions of z_s and z_t with multi-kernel MMD (all moments) and CORAL (second-order, cross-feature correlation).
- **Keep target features informative** (λ^*): masked reconstruction of target windows, which is label-free.
- **Do not drift or forget**: an L2-SP penalty pulls the weights towards the source solution θ_S .

5.1 Algorithm

1. Initialise $\theta \leftarrow \theta_S$ (trained source model); store a frozen copy θ_S .
2. For each of $E = 10$ epochs, for $\lceil |D_T| / 256 \rceil$ steps: sample a source batch (x_s, y_s) and a target batch x_t of 256 windows each.
3. Forward both batches: $\hat{y}_s, z_s = f_{\theta}(x_s)$ and $z_t = \text{enc}_{\theta}(x_t)$; compute the loss of Section 6; take one AdamW step (lr $3e-4$, gradient clipping 1.0).
4. After every epoch log each loss term, the feature MMD² between source-validation and target-validation latents, and the source-validation MSE. Target test MSE is logged only for drawing curves; it is never used for selection.
5. Return the final θ' (no early stopping on target data).

6. Stage 5 — Loss function design

$$L_{\text{CARE}} = L_{\text{fcst}} + \lambda_M \cdot \text{MMD}^2_k(z_s, z_t) + \lambda_C \cdot \text{CORAL}(z_s, z_t) + \lambda_R \cdot L_{\text{mask}}(x_t) + \lambda_A \cdot \|\theta - \theta_S\|^2$$

Forecasting loss. $L_{\text{fcst}} = (1/(B \cdot H \cdot C)) \sum \|f_{\theta}(x_s) - y_s\|^2$, the mean squared error on standardised values.

Multi-kernel MMD [4,5]. With Gaussian kernels $k_{\sigma}(a,b) = \exp(-\|a-b\|^2/\sigma)$ for $\sigma \in \{\frac{1}{4}, \frac{1}{2}, 1, 2, 4\} \times$ the median pairwise squared distance (median heuristic):

$$\text{MMD}^2_k = E[k(z_s, z_{s'})] + E[k(z_t, z_{t'})] - 2 E[k(z_s, z_t)]$$

CORAL [3] matches second-order statistics, i.e. how latent features co-vary, which for multivariate series reflects inter-variable structure:

$$\text{CORAL} = \|\text{Cov}(z_s) - \text{Cov}(z_t)\|_F^2 / (4 d^2)$$

Masked reconstruction. A random 30% of the time steps of x_t are zeroed (after RevIN); the reconstruction head predicts the full window from z and the squared error is averaged over the masked positions only. This keeps the encoder sensitive to the actual structure of target data, so that alignment cannot collapse the features.

Elastic anchor (L2-SP) [6]: the squared Euclidean distance between current and source weights. It limits forgetting of Domain 1 and makes negative transfer less likely.

Weights. $\lambda_M = \lambda_C = \lambda_R = 1$ and $\lambda_A = 0.01$, identical across all benchmarks; they were fixed a priori and not tuned on target labels. A post-hoc sensitivity study is reported in Section 8.5.

7. Stage 6 — Performance metrics

7.1 Forecast accuracy

- **MSE and MAE** on per-domain standardised values, averaged over H steps and C variables: comparable across variables with different units, and standard in the forecasting literature.
- **MASE** [18]: forecast MAE divided by the in-sample MAE of the seasonal-naive forecast on the same input window (season 24 for hourly data, 1 for daily data). Values below 1 beat the naive forecast; it is scale-free and comparable across domains.
- **Key-variable RMSE / MAE in physical units**: temperature in °C (climate), daily range in % (finance) and oil temperature in °C (energy); interpretable for end users and shown in the interface.

7.2 Adaptation efficiency

$$\text{GCR} = (\text{E}_{\text{source-only}} - \text{E}_{\text{method}}) / (\text{E}_{\text{source-only}} - \text{E}_{\text{oracle}})$$

- **Gap-closure ratio (GCR)**: the share of the error gap between the un-adapted model and a model fine-tuned with target labels that a label-free method closes. 1 means oracle-level; 0 means no gain; negative values indicate negative transfer.
- **Feature discrepancy**: MMD^2 between source and target latents on held-out windows, which checks whether alignment actually happened.
- **Forgetting %**: relative change of Domain 1 test MSE after adaptation.
- **Stability**: mean $\pm$ standard deviation over three seeds.

8. Stage 7 — Experiments

8.1 Protocol

- All adaptation methods start from the **same source checkpoint** per seed and run for the same number of epochs with the same optimiser.
- **Baselines:** seasonal naive; source-only; DANN [2] (gradient reversal, weight 0.1 with the standard λ schedule); Deep CORAL [3]; MMD [4]. **Ablations:** CARE-DA without alignment, without SSL, without anchor.
- **Label-using references** (not UDA): target-only (trained from scratch on labeled Domain 2) and fine-tune oracle (source model fine-tuned on labeled Domain 2 with target-validation early stopping).
- Seeds 0, 1, 2; CPU only (PyTorch).

8.2 Main results

Table 2 lists target-domain (Domain 2) test metrics. Standard deviations are over seeds.

Climate: Hilly (Shimla) → Plain (Delhi)

Method	MSE	Δ vs src-only	MASE	temperature_2m RMSE	Feat. MMD ²	GCR	Forget %
Seasonal naive	0.351	+38.5%	1.030	1.553	—	—	—
Source-only	0.254 $\pm$ 0.004	—	1.030	1.768	0.078	0.00	0.0
DANN	0.268 $\pm$ 0.017	+5.5%	1.059	1.718	1.859	-0.64	2.9
Deep CORAL	0.272 $\pm$ 0.007	+7.1%	1.047	1.690	0.048	-0.83	6.2
MMD	0.234 $\pm$ 0.003	-7.9%	0.918	1.387	0.042	0.92	-1.7
CARE-DA – align	0.268 $\pm$ 0.008	+5.8%	1.074	1.993	0.449	-0.68	-1.2
CARE-DA – SSL	0.234 $\pm$ 0.003	-7.8%	0.932	1.420	0.032	0.91	-2.1
CARE-DA – anchor	0.244 $\pm$ 0.002	-3.8%	0.982	1.564	0.041	0.44	-1.7
CARE-DA (ours)	0.245 $\pm$ 0.002	-3.6%	0.992	1.649	0.016	0.42	-1.3
Target-only*	0.240 $\pm$ 0.003	-5.3%	0.947	1.433	0.183	0.62	22.9
Fine-tune oracle*	0.232 $\pm$ 0.004	-8.5%	0.929	1.411	0.120	1.00	14.1

Table 2a. Climate target test set. * uses Domain 2 labels. GCR: gap-closure ratio computed from seed-averaged MSEs (source-only = 0, fine-tune oracle = 1). Forget %: change of Domain 1 test MSE.

Energy: Transformer station ETTh1 → ETTh2

Method	MSE	Δ vs src-only	MASE	OT RMSE	Feat. MMD ²	GCR	Forget %
Seasonal naive	0.225	+30.0%	1.025	4.770	—	—	—
Source-only	0.173 $\pm$ 0.004	—	0.958	4.654	0.183	0.00	0.0
DANN	0.217 $\pm$ 0.020	+25.7%	1.084	5.649	1.448	-8.57	12.6
Deep CORAL	0.192 $\pm$ 0.006	+11.4%	1.011	5.233	0.085	-3.81	12.5
MMD	0.165 $\pm$ 0.004	-4.4%	0.931	4.488	0.040	1.46	-3.0
CARE-DA – align	0.181 $\pm$ 0.009	+4.5%	0.986	4.785	0.500	-1.50	-0.8
CARE-DA – SSL	0.162 $\pm$ 0.004	-6.4%	0.917	4.352	0.065	2.14	-5.6
CARE-DA – anchor	0.173 $\pm$ 0.004	-0.1%	0.954	4.695	0.046	0.04	0.2
CARE-DA (ours)	0.167 $\pm$ 0.004	-3.3%	0.938	4.525	0.038	1.10	-3.6
Target-only*	0.177 $\pm$ 0.005	+2.6%	0.957	4.539	0.135	-0.88	31.3
Fine-tune oracle*	0.168 $\pm$ 0.001	-3.0%	0.918	4.263	0.123	1.00	6.1

Table 2b. Energy target test set. * uses Domain 2 labels. GCR: gap-closure ratio computed from seed-averaged MSEs (source-only = 0, fine-tune oracle = 1). Forget %: change of Domain 1 test MSE.

Finance: IT stocks → Pharma stocks (NSE)

Method	MSE	Δ vs src-only	MASE	hl_range RMSE	Feat. MMD ²	GCR	Forget %
Seasonal naive	1.024	+82.5%	1.061	1.403	—	—	—
Source-only	0.561 $\pm$ 0.001	—	0.786	1.083	0.029	0.00	0.0
DANN	0.572 $\pm$ 0.003	+2.0%	0.796	1.100	0.024	-0.84	1.0
Deep CORAL	0.573 $\pm$ 0.002	+2.0%	0.796	1.100	0.025	-0.86	1.1
MMD	0.558 $\pm$ 0.002	-0.5%	0.782	1.082	0.016	0.20	-0.9
CARE-DA – align	0.564 $\pm$ 0.002	+0.6%	0.789	1.088	0.031	-0.23	0.1
CARE-DA – SSL	0.558 $\pm$ 0.002	-0.6%	0.783	1.080	0.020	0.24	-0.7
CARE-DA – anchor	0.564 $\pm$ 0.002	+0.5%	0.788	1.087	0.020	-0.22	-0.1
CARE-DA (ours)	0.562 $\pm$ 0.002	+0.1%	0.786	1.084	0.030	-0.03	-0.3
Target-only*	0.551 $\pm$ 0.001	-1.9%	0.776	1.072	0.025	0.78	-0.1
Fine-tune oracle*	0.548 $\pm$ 0.000	-2.4%	0.773	1.067	0.026	1.00	-1.0

Table 2c. Finance target test set. * uses Domain 2 labels. GCR: gap-closure ratio computed from seed-averaged MSEs (source-only = 0, fine-tune oracle = 1). Forget %: change of Domain 1 test MSE.

Change in Domain 2 test MSE relative to the un-adapted source model (mean of 3 seeds; * uses labels)

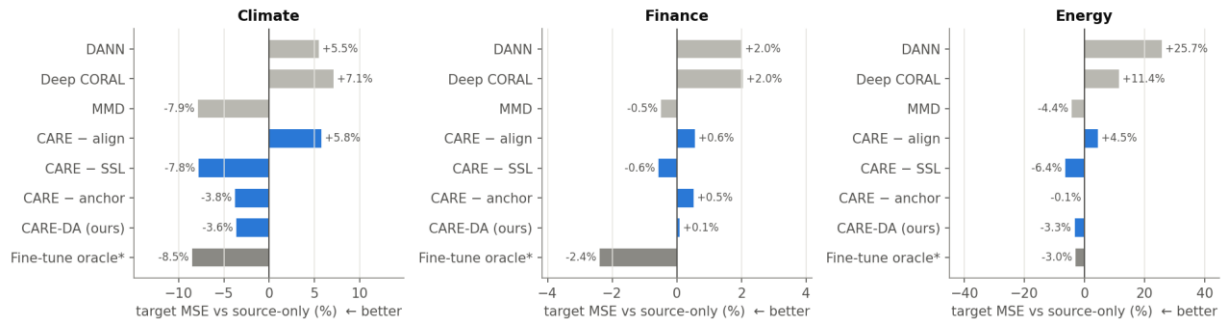


Figure 4. Change of Domain 2 test MSE relative to the un-adapted source model (mean of 3 seeds).

8.3 Findings

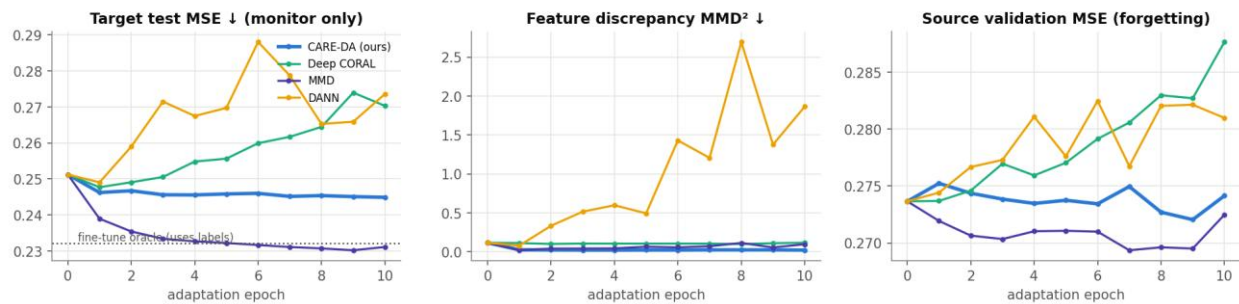
- **Label-free adaptation helps where the shift is real.** On climate (hilly → plain) the best label-free variants (MMD, and CARE-DA without SSL) reduce target MSE by 7.8–7.9%, close to the 8.5% of supervised fine-tuning, and the temperature RMSE drops from 1.77 °C (source-only) to 1.39 °C with MMD. On energy, CARE-DA – SSL reduces MSE by 6.4%, more than the fine-tune oracle (3.0%) and the target-only model (which is 2.6% worse than source-only), showing that the source knowledge plus alignment is worth more than the limited target labels.
- **The designed CARE-DA helps, but less than its best ablation.** Full CARE-DA improves climate by 3.6% and energy by 3.3% and never causes negative transfer, but removing the masked-reconstruction term improves all three benchmarks. The post-hoc sensitivity study confirms a monotone trend: target error increases as λ_R grows from 0 to 1 on every benchmark.
- **Alignment is the working core.** Removing the MMD + CORAL terms (CARE – align) turns every benchmark into negative transfer (+5.8%, +4.5%, +0.6%), even though the anchor and SSL are kept.
- **Adversarial and covariance-only alignment are unstable for regression.** DANN (+5.5%, +25.7%, +2.0%) and Deep CORAL (+7.1%, +11.4%, +2.0%) are worse than doing nothing. DANN's latent discrepancy grows during training (MMD² $\approx$ 1.9 on climate), i.e. the min-max game does not converge with this small encoder.

- **Smaller feature discrepancy does not imply better forecasts.** Full CARE-DA reaches the lowest latent MMD² on climate (0.016 vs 0.078 before adaptation) but forecasts worse than MMD-only (MMD² 0.042). Alignment must preserve forecast-relevant information, which is exactly the λ^* term of the bound.
- **The anchor matters when the shift is strong.** On energy, removing the anchor removes almost the whole gain (−0.1% vs −3.3%); on climate and finance its effect is small.
- **Label-free adaptation does not forget Domain 1.** CARE-DA and MMD change Domain 1 test MSE by −1% to −4% (they slightly improve it), whereas supervised fine-tuning on Domain 2 increases Domain 1 error by 14% (climate) and 6% (energy).
- **Finance: little to gain.** Even the oracle only improves finance by 2.4%; returns are close to unpredictable and the IT/pharma feature distributions differ mainly in level, which per-domain scaling and RevIN already remove. We report this as a genuine negative result.

8.4 Adaptation behaviour (logs and plots)

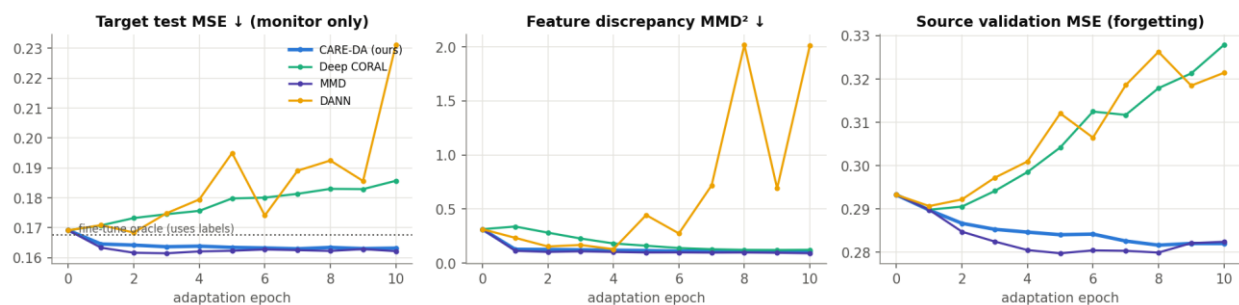
Per-epoch logs of every loss term, the latent MMD², source-validation MSE and (for monitoring only) target test MSE are stored in results/<benchmark>/histories.json and train_log.txt.

Climate: Hilly (Shimla) -> Plain (Delhi)



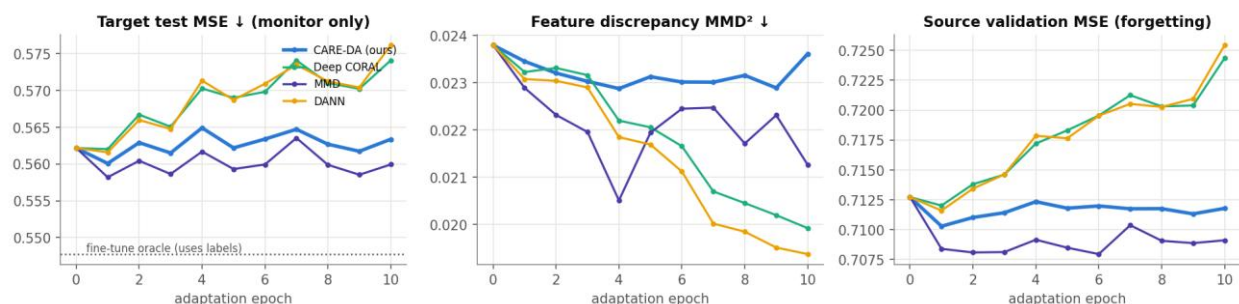
Adaptation curves, Climate (seed 0).

Energy: Transformer station ETTh1 -> ETTh2

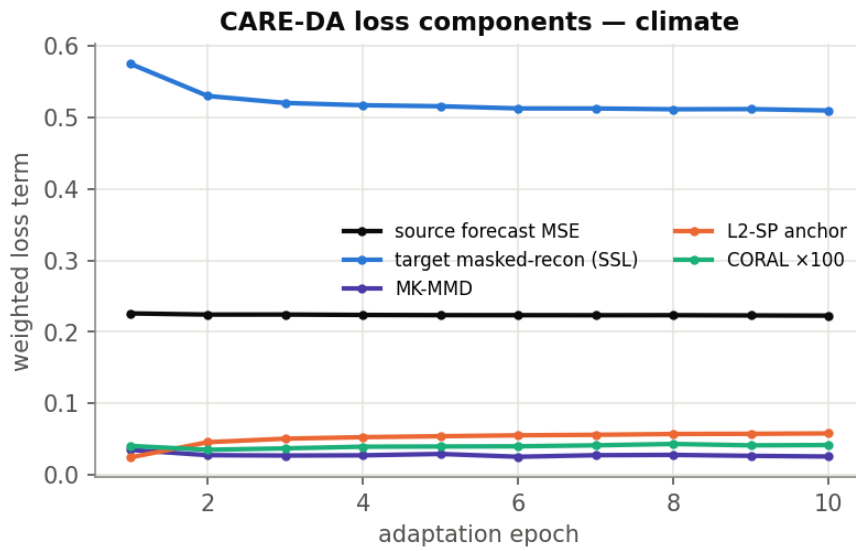


Adaptation curves, Energy (seed 0).

Finance: IT stocks -> Pharma stocks (NSE)

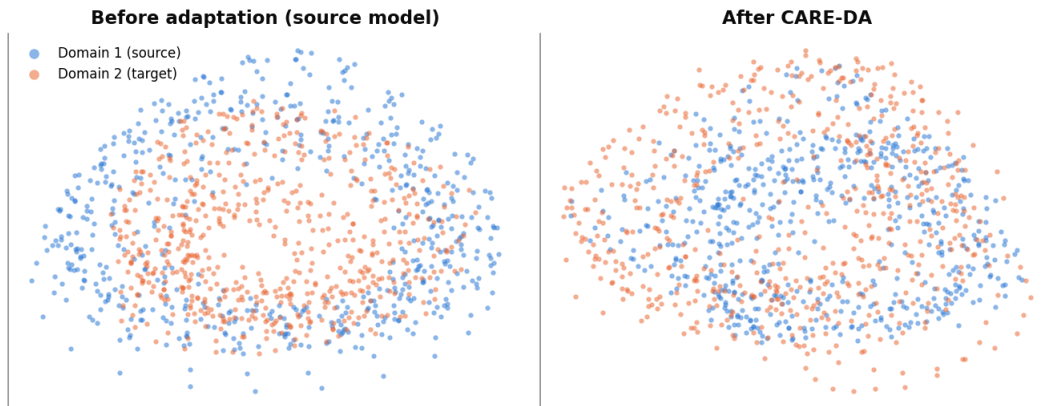


Adaptation curves, Finance (seed 0).



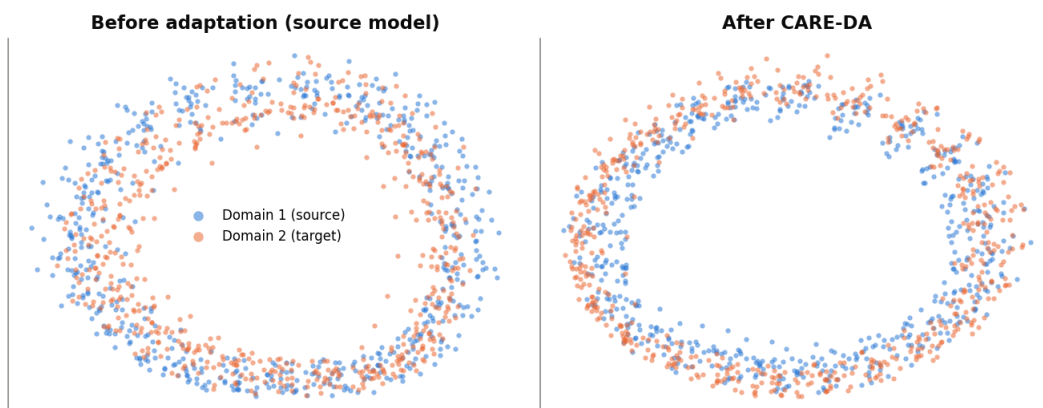
Weighted CARE-DA loss components during adaptation (climate, seed 0).

Latent space (PCA of encoder features) — Energy: Transformer station ETTh1 -> ETTh2

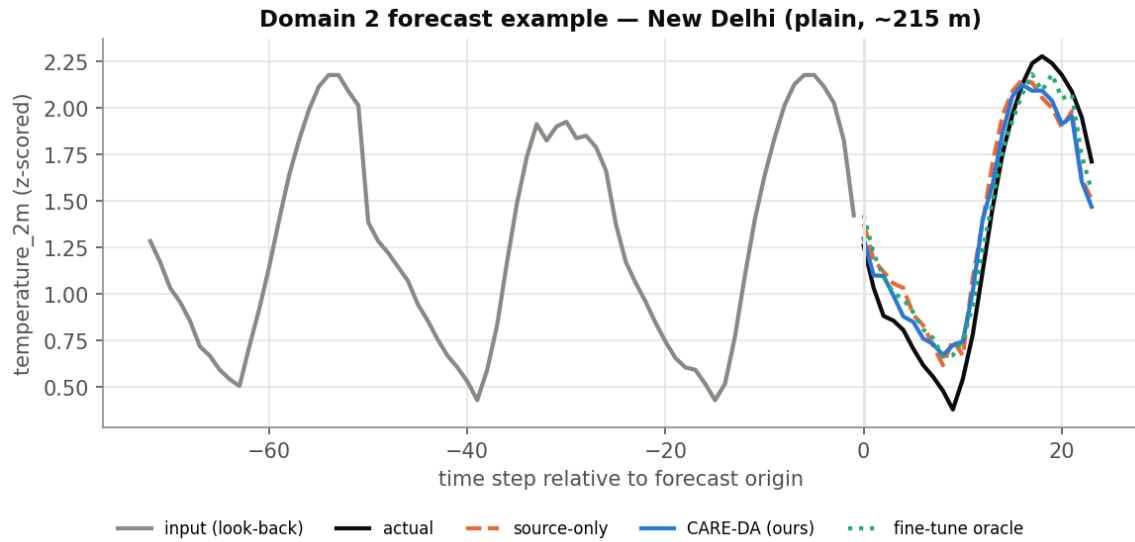


PCA of encoder latents for source (blue) and target (orange) windows before and after CARE-DA (energy).

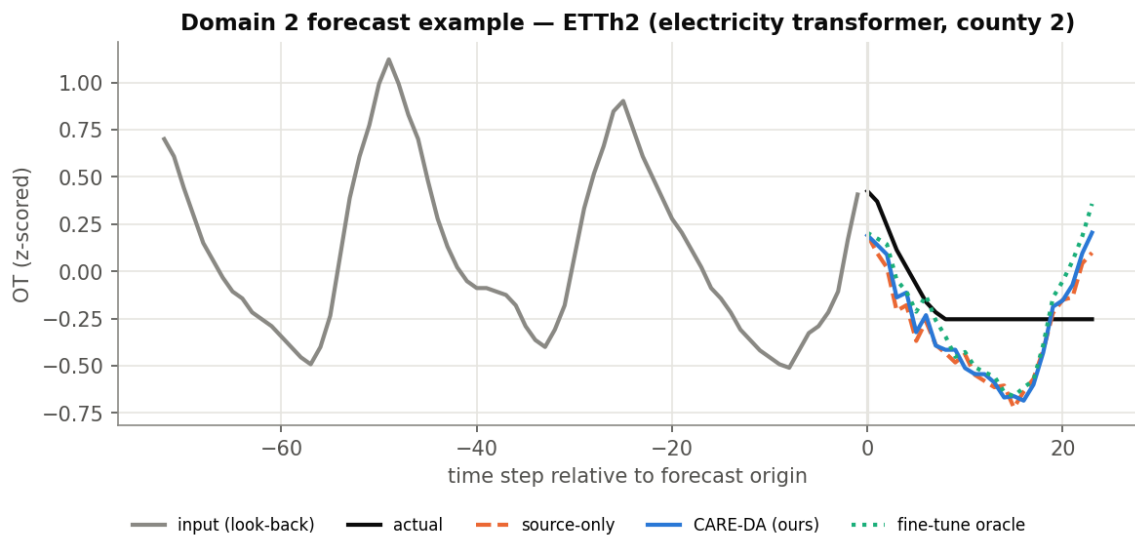
Latent space (PCA of encoder features) — Climate: Hilly (Shimla) -> Plain (Delhi)



Same for the climate benchmark.

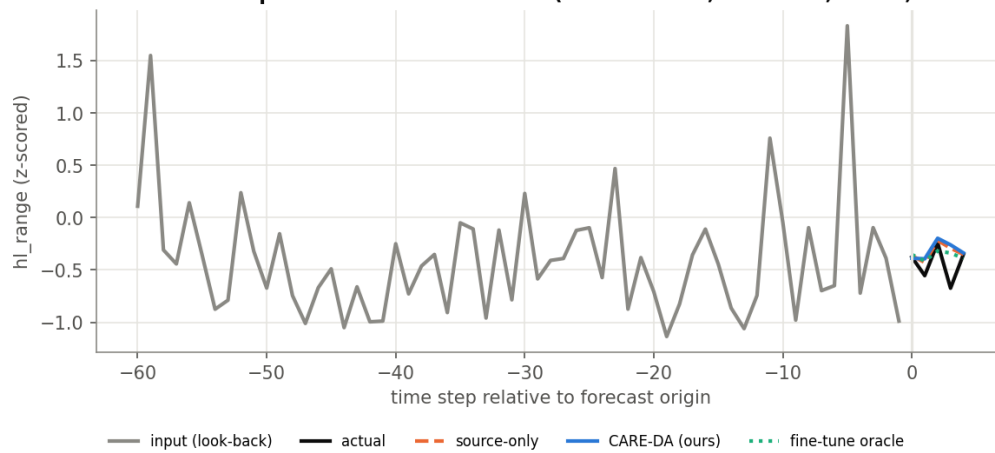


Qualitative Domain 2 forecast, Climate (key variable, z-scored).



Qualitative Domain 2 forecast, Energy (key variable, z-scored).

Domain 2 forecast example — NSE Pharma basket (SUNPHARMA, DRREDDY, CIPLA, DIVISLAB, LUPIN)

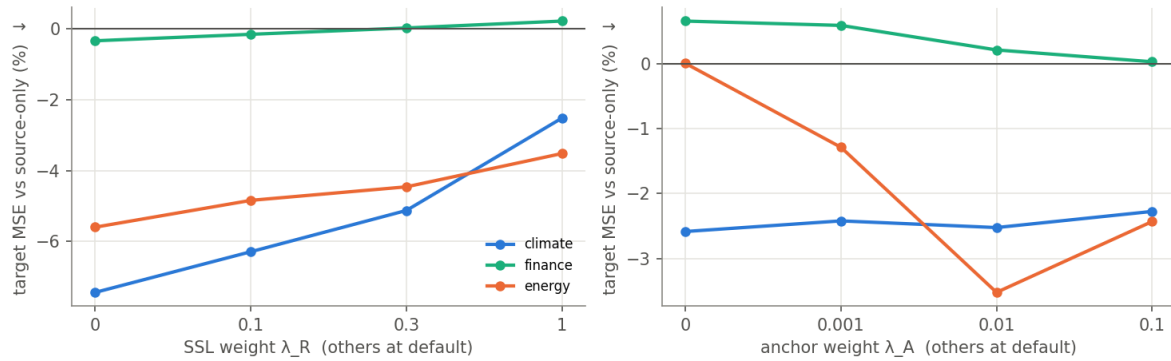


Qualitative Domain 2 forecast, Finance (key variable, z-scored).

8.5 Post-hoc sensitivity

To understand the effect of the two regularisers, we varied λ_R (SSL) and λ_A (anchor) one at a time from the seed-0 source model. These runs were performed **after** fixing the main configuration and are reported for transparency; they are not used to select the model in Table 2.

Post-hoc sensitivity of CARE-DA (seed 0; not used for model selection)



Relative change of target MSE vs source-only as a function of λ_R and λ_A .

λ_R (masked reconstruction): target MSE is lowest at $\lambda_R = 0$ on all three benchmarks and increases monotonically up to $\lambda_R = 1$ (climate 0.233 $\rightarrow$ 0.245, energy 0.160 $\rightarrow$ 0.163, finance 0.560 $\rightarrow$ 0.563). Our interpretation is that reconstructing masked input values rewards features that encode fine-grained input detail, which competes with the forecast-relevant, domain-invariant features that alignment tries to build.

λ_A (anchor): on energy the default $\lambda_A = 0.01$ gives the lowest error (0.169 without the anchor vs 0.163); on climate and finance the effect is within $\pm 0.5\%$. The anchor therefore acts as a safety term against drifting, which matters most when the domain shift is large.

9. Stage 8 — Interface methodology

The interface (app/app.py) is a Streamlit web application that satisfies deliverables 2–4: a user uploads multivariate time-varying data from Domain 1 or Domain 2 as CSV and sees the input, the actual future and the forecast, together with goodness-of-fit metrics.

9.1 Design

- **Model registry:** for each benchmark, models/ holds source_model.pt (Domain 1), adapted_model.pt (Domain 2, CARE-DA) and meta.json with variable names, L, H, seasonal period and both domain scalers. Models are loaded once and cached.
- **Input handling:** the CSV must contain a date column and the benchmark's variables; for finance, raw Yahoo-style OHLCV is accepted and converted with the same feature code used in training (ohlcvcv_to_features), avoiding train/serve skew. Missing values are interpolated; missing columns or too-short files give clear error messages.
- **Domain routing:** Domain 1 data are scaled with the source scaler and forecast by the source model; Domain 2 data use the target scaler and the adapted model, and the source-only forecast can be overlaid to make the effect of adaptation visible.
- **Visualisation:** interactive Plotly panels, one per selected variable, show the look-back input (grey), actual future (black), adapted forecast (blue) and source-only forecast (orange, dashed), with a unified hover tooltip. A slider selects the forecast origin.
- **Metrics:** for the selected window: MSE, MAE, MASE and key-variable RMSE; for the whole file: a rolling evaluation over non-overlapping windows comparing the deployed model with source-only and seasonal-naive forecasts, plus the relative change in MSE due to adaptation.
- **Export and benchmark view:** forecasts can be downloaded as CSV; a second tab shows the benchmark tables and figures from Section 8.

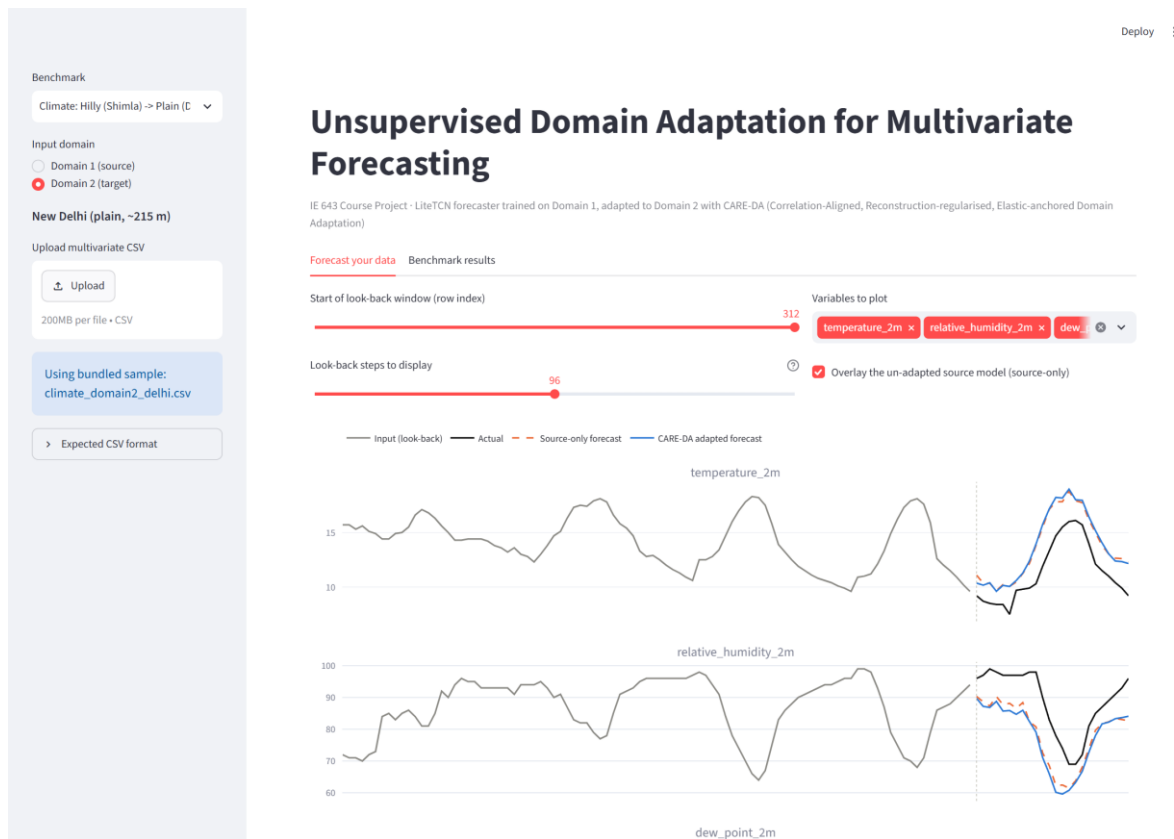


Figure. The interface with Domain 2 (New Delhi) data.

9.2 How to run

```
pip install -r requirements.txt  
streamlit run app/app.py
```

Sample inputs from the held-out test period of every domain are provided in `sample_inputs/` and are used automatically when no file is uploaded.

10. Discussion and limitations

- **Normalisation first.** Per-domain z-scoring and RevIN remove most of the level/scale (marginal) shift; for example, the 13 °C mean temperature difference between Shimla and Delhi is invisible to the encoder. The remaining shift concerns dynamics and cross-variable structure, which is what feature alignment addresses.
- **Why our SSL hypothesis failed.** Test-time training works for classification when the pretext and main tasks share features. For forecasting, masked reconstruction of the look-back window is a different objective from predicting the future, and with a shared 128-d bottleneck it pulls capacity away from forecasting. A forecasting-consistent pretext, for example predicting the last part of the target window from the first part, is a better candidate.
- **Model selection without labels** remains the key open problem of UDA. We used a fixed number of adaptation epochs; DANN's instability shows that an unsupervised selection criterion (e.g. reverse validation or source-risk + discrepancy) is needed before adversarial methods can be used safely.
- **Limitations.** Three seeds per setting; one source/target pair per benchmark; hyper-parameters of baselines were not tuned per benchmark (same as ours); the ETT oracle gap is small (3%), which makes gap-closure ratios unstable. We therefore report relative MSE change as the main cross-benchmark metric and gap-closure computed from seed-averaged errors.
- **Deployment choice.** The interface ships the CARE-DA model exactly as designed (seed 0), so the demo matches the pre-registered method. Switching to the stronger CARE-DA – SSL variant is planned as CARE-DA v2 after unsupervised model-selection experiments.

11. Future work and roadmap

- Verify the three datasets with the TAs; add further target domains (a coastal city, single stocks, more ETT stations) to test robustness of the conclusions.
- CARE-DA v2: drop or replace the masked-reconstruction term with a forecasting-consistent pretext task; add frequency-domain alignment (RAINCOAT-style) for strongly periodic data.
- Unsupervised hyper-parameter and checkpoint selection (reverse validation, source risk + discrepancy) so that λ can be chosen without target labels.
- Few-shot extension: 1–5% labeled target windows combined with CARE-DA, and a comparison with DAF-style attention sharing.
- Interface: batch upload of multiple files, per-variable metric breakdown and uncertainty bands; final stage-wise report and presentation.

12. Code, documentation and reproducibility

Path	Purpose
src/config.py	Benchmark definitions (domains, variables, L, H, splits), paths
src/data_download.py	Downloads and curates all datasets
src/data.py	Splitting, per-domain scaling, windowing, batching
src/model.py	LiteTCN-Forecaster and RevIN
src/losses.py	MK-MMD, CORAL, DANN discriminator with gradient reversal, L2-SP
src/metrics.py	MSE, MAE, MASE, key-variable errors, gap-closure ratio
src/train.py	Source training, CARE-DA and baseline adaptation
src/run_experiments.py	Full benchmark for one dataset (all methods, seeds, logs, checkpoints)
src/sensitivity.py, src/plots.py	Sensitivity study; all figures
app/app.py	Streamlit interface
models/, results/, sample_inputs/	Trained models + scalers; logs, metrics, figures; demo CSVs

Dependencies: Python ≥ 3.10 , torch, numpy, pandas, matplotlib, requests, yfinance, streamlit, plotly (requirements.txt). **Commands:**

```
python -m src.data_download
python -m src.run_experiments --dataset climate --seeds 0 1 2    (and energy, finance)
python -m src.sensitivity --dataset climate    (and energy, finance)
python -m src.plots
streamlit run app/app.py
```

References

- [1] Shai Ben-David, John Blitzer, Koby Crammer, Alex Kulesza, Fernando Pereira and Jennifer Wortman Vaughan. A theory of learning from different domains, *Machine Learning*, 2010. <https://doi.org/10.1007/s10994-009-5152-4>
- [2] Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario Marchand and Victor Lempitsky. Domain-Adversarial Training of Neural Networks, *Journal of Machine Learning Research*, 2016. <https://arxiv.org/abs/1505.07818>
- [3] Baochen Sun and Kate Saenko. Deep CORAL: Correlation Alignment for Deep Domain Adaptation, *ECCV Workshops*, 2016. <https://arxiv.org/abs/1607.01719>
- [4] Mingsheng Long, Yue Cao, Jianmin Wang and Michael I. Jordan. Learning Transferable Features with Deep Adaptation Networks, *ICML*, 2015. <https://arxiv.org/abs/1502.02791>
- [5] Arthur Gretton, Karsten M. Borgwardt, Malte J. Rasch, Bernhard Schölkopf and Alexander Smola. A Kernel Two-Sample Test, *Journal of Machine Learning Research*, 2012. <https://jmlr.org/papers/v13/gretton12a.html>
- [6] Xuhong Li, Yves Grandvalet and Franck Davoine. Explicit Inductive Bias for Transfer Learning with Convolutional Networks, *ICML*, 2018. <https://arxiv.org/abs/1802.01483>
- [7] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros and Moritz Hardt. Test-Time Training with Self-Supervision for Generalization under Distribution Shifts, *ICML*, 2020. <https://arxiv.org/abs/1909.13231>
- [8] Xiaoyong Jin, Youngsuk Park, Danielle C. Maddix, Hao Wang and Yuyang Wang. Domain Adaptation for Time Series Forecasting via Attention Sharing, *ICML*, 2022. <https://arxiv.org/abs/2102.06828>
- [9] Garrett Wilson, Janardhan Rao Doppa and Diane J. Cook. Multi-Source Deep Domain Adaptation with Weak Supervision for Time-Series Sensor Data, *KDD*, 2020. <https://arxiv.org/abs/2005.10996>
- [10] Ruichu Cai, Jiawei Chen, Zijian Li, Wei Chen, Keli Zhang, Junjian Ye, Zhuozhang Li, Xiaoyan Yang and Zhenjie Zhang. Time Series Domain Adaptation via Sparse Associative Structure Alignment, *AAAI*, 2021. <https://arxiv.org/abs/2012.11797>
- [11] Yilmazcan Ozyurt, Stefan Feuerriegel and Ce Zhang. Contrastive Learning for Unsupervised Domain Adaptation of Time Series, *ICLR*, 2023. <https://arxiv.org/abs/2206.06243>
- [12] Huan He, Owen Queen, Teddy Koker, Consuelo Cuevas, Theodoros Tsiligkaridis and Marinka Zitnik. Domain Adaptation for Time Series Under Feature and Label Shifts, *ICML*, 2023. <https://arxiv.org/abs/2302.03133>
- [13] Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi and Jaegul Choo. Reversible Instance Normalization for Accurate Time-Series Forecasting against Distribution Shift, *ICLR*, 2022. <https://openreview.net/forum?id=cGDAkQo1C0p>
- [14] Ailing Zeng, Muxi Chen, Lei Zhang and Qiang Xu. Are Transformers Effective for Time Series Forecasting?, *AAAI*, 2023. <https://arxiv.org/abs/2205.13504>
- [15] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong and Jayant Kalagnanam. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers, *ICLR*, 2023. <https://arxiv.org/abs/2211.14730>
- [16] Shaojie Bai, J. Zico Kolter and Vladlen Koltun. An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling, *arXiv preprint*, 2018. <https://arxiv.org/abs/1803.01271>
- [17] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong and Wancai Zhang. Informer: Beyond Efficient Transfomer for Long Sequence Time-Series Forecasting, *AAAI*, 2021. <https://arxiv.org/abs/2012.07436>
- [18] Rob J. Hyndman and Anne B. Koehler. Another look at measures of forecast accuracy, *International Journal of Forecasting*, 2006. <https://doi.org/10.1016/j.ijforecast.2006.03.001>
- [19] Open-Meteo. <https://open-meteo.com/en/docs/historical-weather-api>. Accessed on: 18th September, 2026.
- [20] Ran Aroussi. <https://github.com/ranaroussi/yfinance>. Accessed on: 18th September, 2026.
- [21] Haoyi Zhou. <https://github.com/zhouhaoyi/ETDataset>. Accessed on: 18th September, 2026.